Week 14
(WAD & AWP)

JSON WEB TOKEN

As we know tokens are generic **authentication credentials or authorization tokens** used to **authenticate users or authorize access to resources** in web applications.

Tokens refer to a broad category of authentication credentials, whereas **JWT is a specific format or implementation of tokens**.

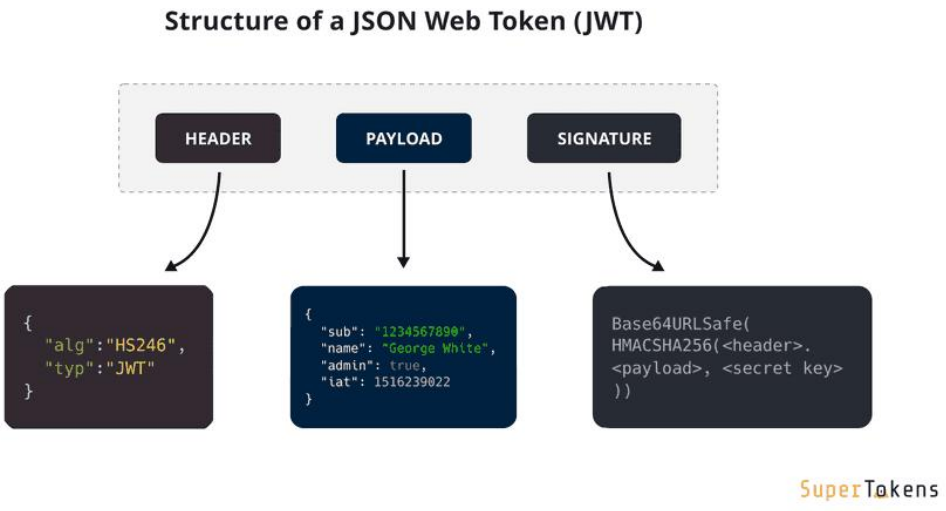
JWT (JSON Web Token)

Definition: JWT is a **specific type of token format** defined by the JSON Web Token standard (RFC 7519). It is a compact and self-contained means of transmitting information between parties as a JSON object.

Structure: JWTs consist of three parts: **header, payload, and signature**. They are typically encoded and signed using cryptographic algorithms.

Usage: JWTS are commonly used for authentication and authorization in web applications and APIs. They can store user claims, such as user ID, roles, permissions, and custom data, in a secure and portable format.

Statelessness: JWTS are stateless, meaning the server does not need to store session information. This makes them suitable for distributed architectures and scalable systems.



JWT Structure

A JWT is composed of three sections separated by dots (.), following the format header.payload. signature.

1. Header: Contains metadata about the type of token and the cryptographic algorithms used to secure it. It typically consists of two parts:

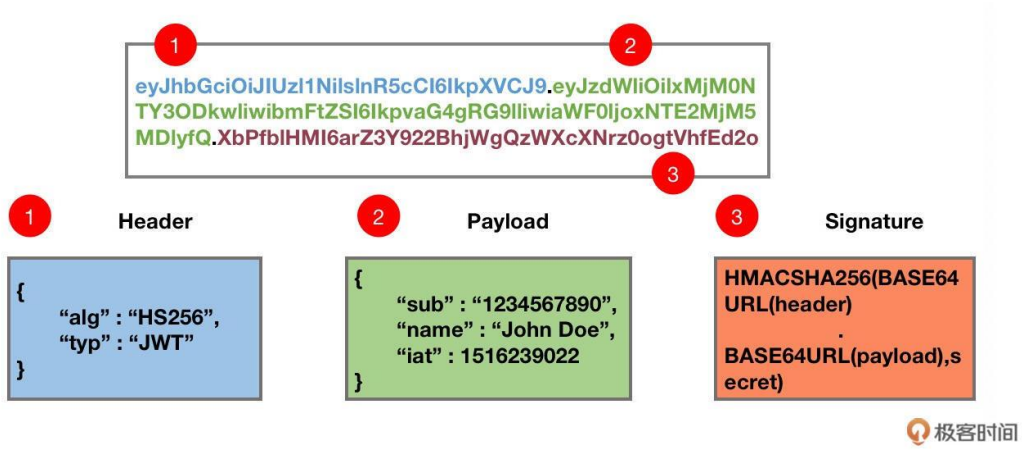
- **Typ (Type):** Specifies the type of token, usually set to "JWT".

- **Alg (Algorithm):** Indicates the cryptographic algorithm used to sign the token, such as HMAC, SHA256 or RSA.

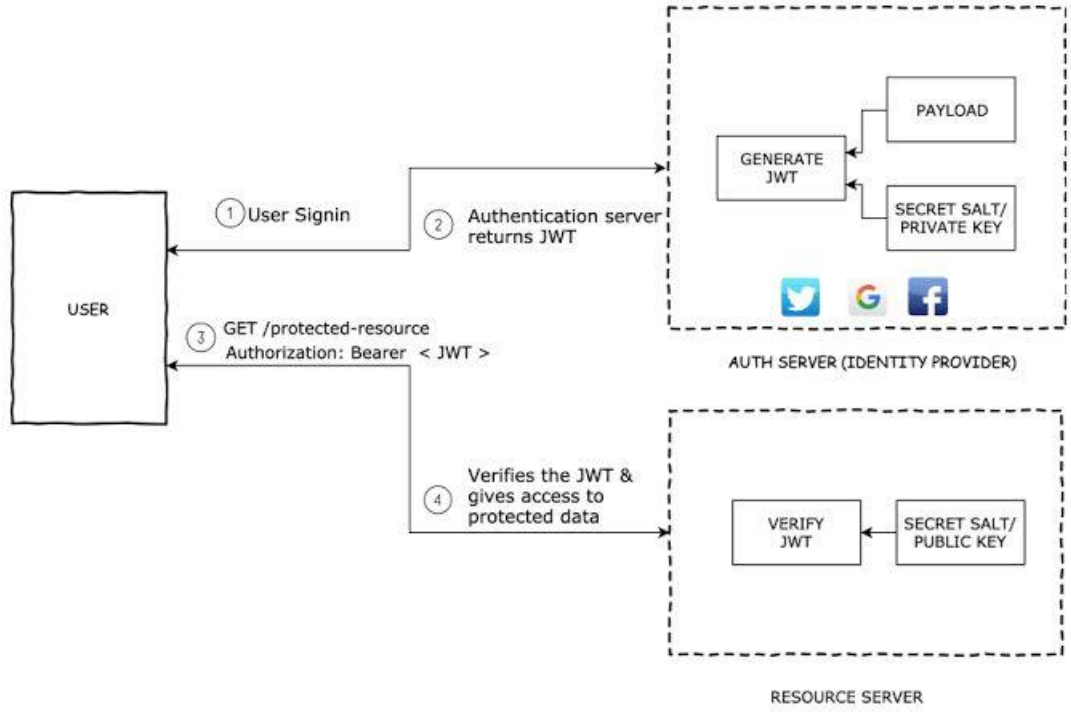
2. Payload: Contains the claims or statements about the subject (user) and any additional data. It consists of a set of claims that represent assertions about the user, such as their identity, roles, or permissions. Claims are categorized into three types:

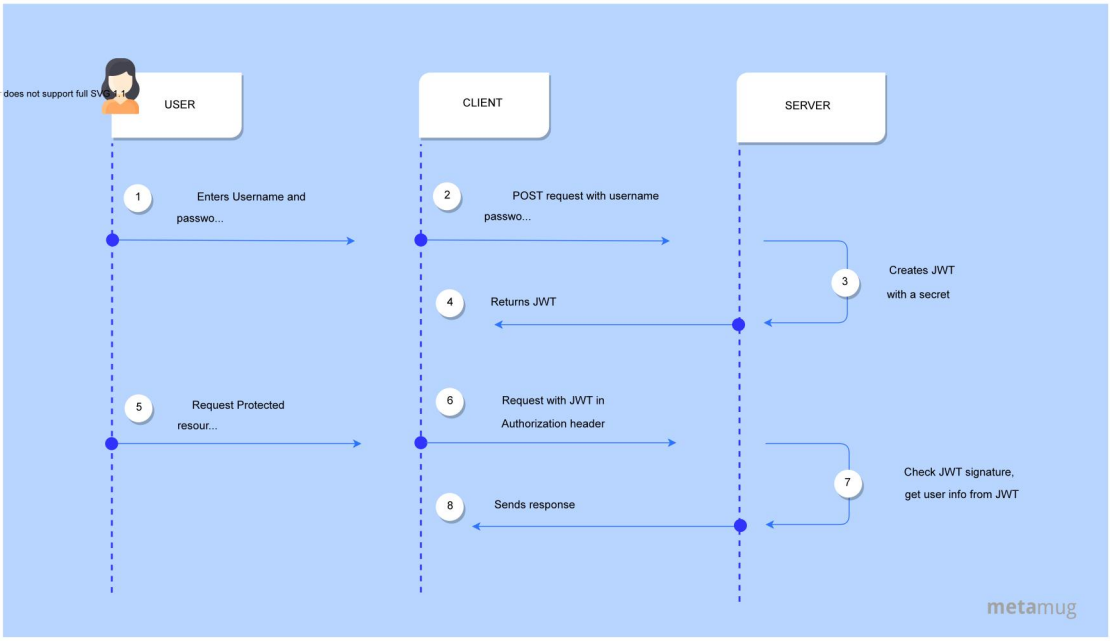
- **Reserved Claims:** Predefined claims standardized by the JWT specification, such as **iss** (issuer), **sub** (subject), **aud** (audience), **exp** (expiration time), and **iat** (issued at).
- **Public Claims:** Custom claims defined by the application developer to convey information about the user.
- **Private Claims:** Custom claims agreed upon by parties that exchange JWTS, not registered or standardized.

3. Signature: Verifies the integrity of the token and ensures that it has not been tampered with during transmission. It's created by taking the encoded header, encoded payload, a secret (for HMAC algorithms), and applying the specified algorithm to generate the signature.



Authentication Flow:





JWT Functions

Certainly! In the context of JSON Web Tokens (JWT), **jwt.sign()** and **jwt.verify()** are two crucial functions provided by the **jsonwebtoken** library in Node.js.

jwt.sign():

This function is used to generate a new JWT token based on the provided payload and options.

It takes three parameters:

- **payload:** This is the data you want to include in the token. It can be any JSON object containing user information, metadata, or any other relevant data.

- **secretor PrivateKey:** This is the secret key used to sign the token. It can be a string or a buffer containing a secret cryptographic key.

- **options (optional):** These are additional options that control the behavior of the token generation process, such as expiration time (expiresIn), algorithm (algorithm), and more.

```
//Payload containing user information
const payload = { userID: '123456', username : 'exampleUser'};

//Secret key for signing the Token
const secret_key = 'your_secret_key';

//Generate a new JWT Token
const token = jwt.sign(payload, secret_key, {expiresIn : '1h'} )
```

jwt.verify():

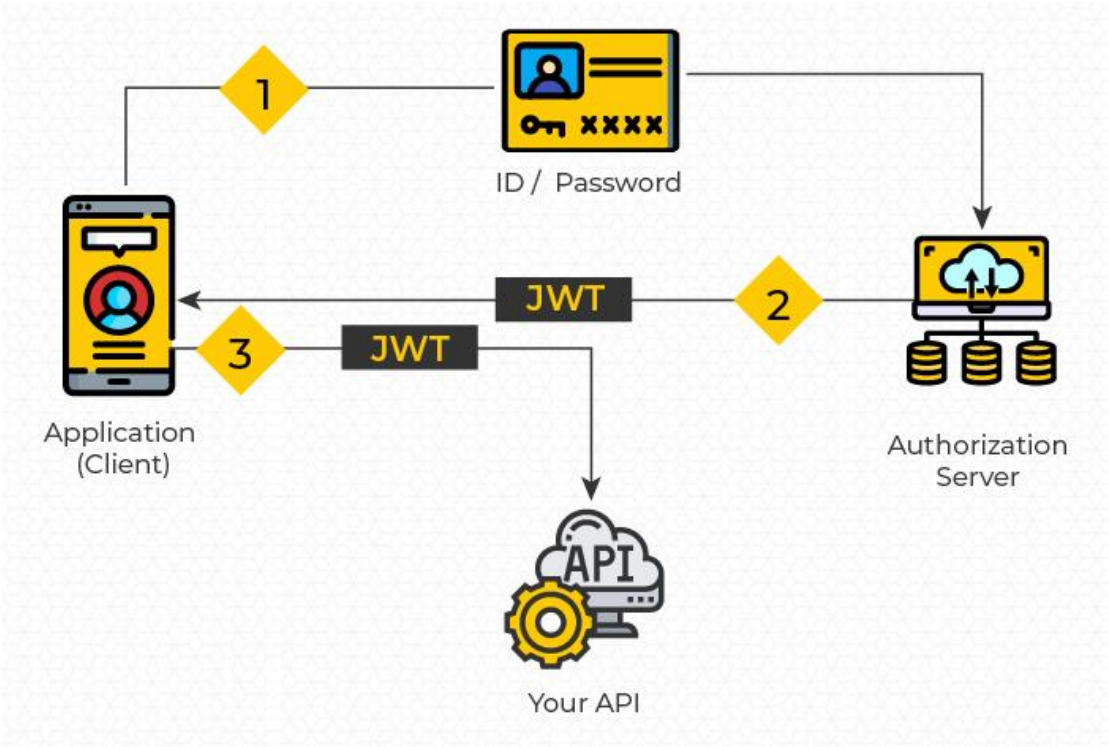
This function is used to verify and decode a JWT token to retrieve the original payload.

It takes three parameters:

- **token:** The JWT token to be verified.

- **secretor PublicKey:** The secret key or public key used to verify the token's signature. If the token was signed using a symmetric algorithm e.g., HMAC), you need to provide the same secret key used to sign the token. If it was signed using an asymmetric algorithm (e.g., RSA), you need to provide the public key corresponding to the private key used for signing.

- **options (optional):** Additional options for verification, such as **algorithms** (algorithms), **audience** (audience), **issuer** (issuer), and more.



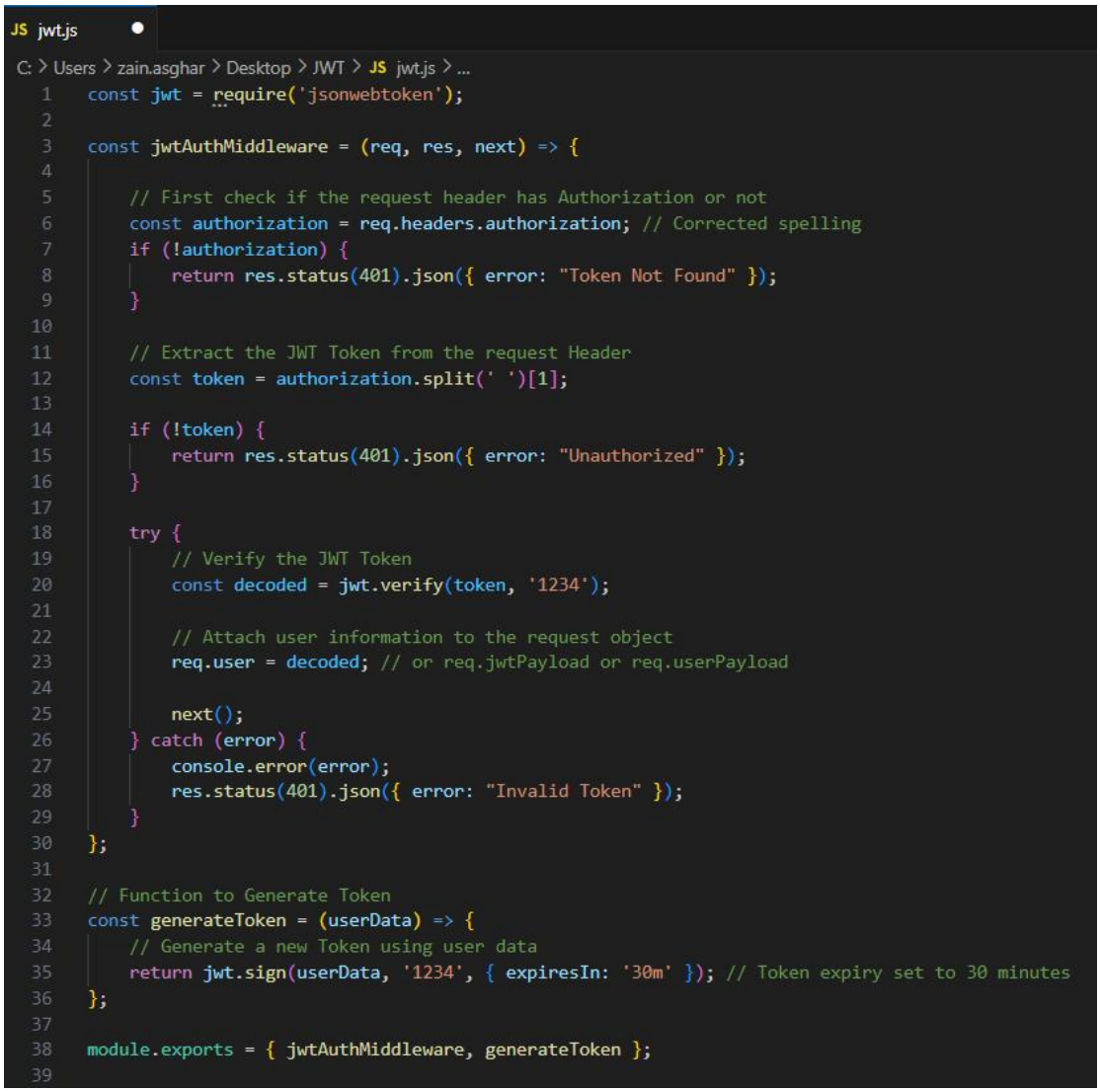
Install JWT

First, you'll need to install the necessary packages for working with JWT. In Node.js, you can use packages like jsonwebtoken to generate and verify JWTS.

npm install jsonwebtoken

jwtAuthMiddleware

Create a **JWT Auth Middleware** function, which is responsible for authentication via Token.



```
JS jwt.js
C: > Users > zain.asghar > Desktop > JWT > JS jwt.js > ...
1   const jwt = require('jsonwebtoken');
2
3   const jwtAuthMiddleware = (req, res, next) => {
4
5       // First check if the request header has Authorization or not
6       const authorization = req.headers.authorization; // Corrected spelling
7       if (!authorization) {
8           return res.status(401).json({ error: "Token Not Found" });
9       }
10
11      // Extract the JWT Token from the request Header
12      const token = authorization.split(' ')[1];
13
14      if (!token) {
15          return res.status(401).json({ error: "Unauthorized" });
16      }
17
18      try {
19          // Verify the JWT Token
20          const decoded = jwt.verify(token, '1234');
21
22          // Attach user information to the request object
23          req.user = decoded; // or req.jwtPayload or req.userPayload
24
25          next();
26      } catch (error) {
27          console.error(error);
28          res.status(401).json({ error: "Invalid Token" });
29      }
30  };
31
32  // Function to Generate Token
33  const generateToken = (userData) => {
34      // Generate a new Token using user data
35      return jwt.sign(userData, '1234', { expiresIn: '30m' }); // Token expiry set to 30 minutes
36  };
37
38  module.exports = { jwtAuthMiddleware, generateToken };
39
```

• We can certainly change the variable name from req.user to req.EncodedData or any other name you prefer. The choice of variable name (EncodedData, userData, etc.) is flexible and depends on your application's conventions or requirements.

• The key aspect is to make sure that the decoded user information is attached to the request object (req) so that it can be easily accessed by other middleware functions or route handlers.

SignUp Route

```
const {jwtAuthMiddleware, generateToken} = require('../jwt')



// Signup Route (Endpoint)
router.post('/signup', async (req, res) => {
    try {
        const data = req.body;

        // Check if both fields are provided
        if (!data.username || !data.password) {
            return res.status(400).json({ error: 'Username and password are required' });
        }

        // Create a new user
        const newUser = new Person(data);

        // Save the user to the database
        const response = await newUser.save();

        const payLoad = {
            id: response.id,
            username: response.username
        }
        //const token = generateToken(response.username)

        const token = generateToken(payLoad)    //Generate token by calling generateToken Function with Payload

        // console.log("Generated Token:", token);

        console.log(response);
        res.status(201).json({ message: 'Signup successful', user: response, token : token });
    } catch (error) {
        // Handle validation errors
        if (error.name === 'ValidationError') {
            res.status(400).json({ error: 'Validation Error', details: error.message });
        }
        // Handle duplicate key error (e.g., unique username constraint)
        else if (error.code === 11000) {
            res.status(400).json({ error: 'Username already exists' });
        }
        // Handle any other errors
        else {
            console.error(error);
            res.status(500).json({ error: 'Internal Server Error' });
        }
    }
});
```

SignIn Route

```
//Login Route

router.post('/login', async (req, res) => {
    try {

        //Extract username and password
        const {username,password} = req.body;

          // Find user by username
          const user = await Person.findOne({ username : username });
          if (!user) {
              return res.status(401).json({ message: 'Incorrect Username.' });
          }

          // Check password
          const isPasswordMatched = await user.comparePassword(password);
          if (!isPasswordMatched){
            return res.status(401).json({ message: 'Incorrect Password.' });
          }

        //Generate Token
        const payLoad = {
            id: user.id,
            username: user.username
        }

        const token = generateToken(payLoad) //Generate token by calling generateToken Function with Payload
        console.log("Generated Token:", token);


        //return Token as response
        res.json({token : token });
    }
    catch (error){

            console.error(error);
            res.status(500).json({ error: 'Internal Server Error' });
    }
});
```